Лабораторна 2
Токенізатори

Конспект із розбором коду та відповідями на питання

Роман Миджин · 23 жовтня 2026

Я порівнюю способи, якими текст перетворюється на токени: слова, символьні n-грами, BPE та WordPiece. Потім дивлюся, скільки токенів вони створюють для англійської, української та грузинської, і порівнюю з готовими кодуваннями GPT.

1. Текст, токен, ID та embedding

Токен - одиниця розбиття: слово, частина слова, число або знак. **ID** - її номер у словнику. Номер не є значенням слова: сусідні ID не означають схожість. **Embedding** - навчений вектор, який модель бере за цим ID.

Шлях даних: **текст → токени → ID → вектори → трансформер**. У лабі я реалізую токенізацію й кодування, а не навчаю мовну модель.

```
def word_tokenize(text):
    """Слова (з апострофами), числа та окремі знаки; нижній регістр."""
    return re.findall(r"[^\W\d_]+(?:['’][^\W\d_]+)*|\d+|[^\w\s]|_",
        text.lower())
```

Регулярний вираз окремо знаходить слова з внутрішніми апострофами, числа та знаки. **lower()** зменшує кількість варіантів, але стирає регістр: «Роман» і «роман» перестають відрізнятися.

Що розібрано далі

2 - словник і OOV; 3 - n-грами та «кіт»; 4 - BPE; 5 - WordPiece-score; 6 - greedy-розбиття; 7 - дані та fertility; 8 - числа; 9 - мови й GPT; 10 - короткі відповіді; 11-15 - докладні нюанси й приклади. Код у сірих блоках узято з поточного notebook, крім явно позначеного прикладу fallback. Перенесення довгих рядків зроблено лише для друку.

2. Словник, невідомі слова та пам’ять

```
def build_word_vocab(texts):
    tokens = sorted({token for text in texts for token in
        word_tokenize(text)})
    return {"<pad>": 0, "<unk>": 1, **{token: i for i, token in
        enumerate(tokens, 2)}}

def encode_words(text, vocab):
    return [vocab.get(token, vocab["<unk>"]) for token in word_tokenize(text)]
```

Що робить кожна частина

set прибирає повтори, **sorted** задає сталий порядок. Через **enumerate(tokens, 2)** звичайні токени отримують ID від 2. Перші два місця зайняті: <pad> = 0 і <unk> = 1.

vocab.get(token, vocab["<unk>"]) повертає ID відомого токена, а для невідомого - 1. <pad> потрібен для доповнення послідовностей до спільної довжини; він не означає невідоме слово.

OOV: out of vocabulary

Якщо навчання бачило «кіт», але не «котик», друге слово стає <unk>. Два різні невідомі слова отримають один ID. Наприклад, невідомі імена в реченнях можуть зникнути в однаковому наборі чисел.

Послівний метод дає коротке представлення знайомих слів, але погано працює з новими формами. «Студент», «студента», «студентами» займають три окремі записи. Підслова дозволяють використовувати частини повторно.

Чому великий словник потребує пам’яті

Embedding-таблиця має **V × d** параметрів: V - кількість токенів, d - довжина вектора. За V = 2000 і d = 128 це 256 000 параметрів; у float32 - приблизно 1 МБ лише для цієї таблиці, без градієнтів та станів оптимізатора.

Компроміс: більший словник може скоротити послідовність, але збільшує таблицю векторів. Менший словник економить місце, проте текст частіше розбивається на дрібні частини.

3. N-грами, Жаккар і слово «кіт»

```
def char_ngrams(word, n=3):
    if not isinstance(n, int) or isinstance(n, bool) or n <= 0:
        raise ValueError("n має бути додатним цілим числом")
    if not word:
        return []
    bounded = "^" + word + "$"
    return [bounded[i:i+n] for i in range(len(bounded) - n + 1)]

def ngram_overlap(word_a, word_b, n=3):
    a, b = set(char_ngrams(word_a, n)), set(char_ngrams(word_b, n))
    return len(a & b) / len(a | b) if a | b else 1.0
```

n-грама - фрагмент із n сусідніх символів. Я додаю ^ і $ для меж слова. Для «кіт», n = 3: **^кі, кіт, іт$**. Фрагменти перекриваються; одна літера може входити в кілька n-грам.

Для непорожнього слова довжини L кількість фрагментів: **max(0, L + 3 - n)**. Два маркери додають 2, а вікно має «довжина рядка - n + 1» положень. Порожнє слово код обробляє окремо.

Шестиграми для «кіт»

«^кіт$» має 5 символів. При n = 6 діапазон range(0) порожній, тому результат [] без винятку. Очікуваний тут варіант - доповнити рядок спеціальними символами до довжини 6. Інша можлива політика - повернути короткий рядок одним токеном, але тоді довжина фрагмента вже не дорівнює n. Правило має бути однаковим при навчанні й застосуванні.

```
bounded = bounded.ljust(n, "~")
```

Це можливий padding після створення bounded, а не зміна в notebook. Тут ~ - один зарезервований символ. ljust додає його лише тоді, коли рядок коротший за n. Детальний приклад і перевірки - на сторінці 11.

Жаккар = |A ∩ B| / |A □ B|: спільні унікальні фрагменти / всі різні. Множини не зберігають порядок і повтори. 0 - нічого спільного, 1 - однакові множини. Дві порожні множини код теж оцінює як 1, тому це може приховати невдале розбиття коротких слів.

Менше n - частіші спільні частини, але менше специфічності. Більше n - більше контексту, але гірше покриття коротких і нових слів.

4. BPE: частоти, злиття, нові слова

```
def most_frequent_pair(vocabulary):
    counts = Counter()
    for symbols, frequency in vocabulary.items():
        for pair in zip(symbols, symbols[1:]):
            counts[pair] += frequency
    return min(counts, key=lambda pair: (-counts[pair], pair)) if counts else
        None

def apply_bpe(word, merges):
    if not word:
        return []
    symbols = tuple(word) + ("</w>",)
    for pair in merges:
        symbols = merge_pair(symbols, pair)
    return list(symbols)
```

У train_bpe я починаю зі списків символів із </w>, рахую пари, зливаю найчастішу й повторюю. **frequency** враховує повторення слова. Ключ **(-counts[pair], pair)** вибирає найбільшу частоту через min; при рівності - пару за лексикографічним порядком.

Приклад

Нехай «кіт» є двічі, «кит» - один раз. Пара (т, </w>) трапляється 3 рази, (к, і) - 2. Перша стає токеном т</w>. Потім перераховую пари вже в змінених послідовностях. Це навчання за підрахунками, без градієнтного спуску.

У **merge_pair** і += 2 пропускає обидві злиті частини, тому входження не перекриваються. Для «ааа» і пари (а, а) результат - [аа, а]. Навчальний код обробляє кожне слово окремо; </w> позначає його кінець.

train_bpe зберігає порядок злиттів. apply_bpe повторює ці правила для нового слова. Історія важлива: раннє правило може створити частину для пізнішого. Нове слово з відомих символів можна подати дрібнішими фрагментами.

Мій BPE символьний і повертає рядки: невідомий символ може залишитися в результаті поза навчальним словником. Це не готовий механізм ID для будь-якого тексту. Byte-level BPE використовує початкові 256 байтів, щоб покрити довільний UTF-8 текст.

5. WordPiece: що означає score

```
def wordpiece_pair_scores(vocabulary):
    token_counts, pair_counts = Counter(), Counter()
    for symbols, count in vocabulary.items():
        for symbol in symbols:
            token_counts[symbol] += count
        for pair in zip(symbols, symbols[1:]):
            pair_counts[pair] += count
    return {pair: count / (token_counts[pair[0]] * token_counts[pair[1]])
            for pair, count in pair_counts.items()}
```

Формула

score(a, b) = freq(a, b) / [freq(a) × freq(b)]. Чисельник - частота сусідньої пари, знаменник - добуток частот її частин. Усі частоти зважені повтореннями слів.

Пара	freq(a,b)	freq(a)	freq(b)	score
A	20	100	50	0.004
B	2	2	2	0.5

BPE обрав би A: у неї більше повторень пари. Мій WordPiece обрав би B: у неї вищий score. Нормування підсилює пару, частини якої частіше пов’язані саме одна з одною відносно їхніх власних частот.

Score - показник для вибору злиття, а не ймовірність слова чи оцінка змісту. Рідкісна пара може мати високий score, але скорочувати мало позицій у всьому корпусі.

Як створюється словник

wordpiece_symbols розділяє слово як т, ##о, ##к, ##е, ##н. Без ## - початок, із ## - продовження. train_wordpiece зберігає стартові частини та [UNK], потім додає злиття з найбільшим score. Старі фрагменти залишаються доступними для нових слів.

Це навчальна схема з лабораторної. У курсі Hugging Face таку схему описано як реконструкцію навчання WordPiece за літературою; її не слід називати точним оприлюдненим алгоритмом навчання оригінального BERT. [1]

6. WordPiece: як розбирається нове слово

```
def apply_wordpiece(word, vocab, unk_token="[UNK]"):
    pieces, start = [], 0
    while start < len(word):
        end = len(word)
        while end > start:
            candidate = ("##" if start else "") + word[start:end]
            if candidate in vocab:
                break
            end -= 1
        if end == start:
            return [unk_token]
        pieces.append(candidate)
        start = end
    return pieces
```

start - початок ще не обробленої частини. **end** спочатку стоїть у кінці слова й рухається вліво. Перший знайдений candidate є найдовшим доступним фрагментом. Для start > 0 додаю ##.

Приклад із тесту

```
vocab = {"[UNK]", "ток", "токен", "##ен", "##ізація"}
apply_wordpiece("токенізація", vocab)
# ["токен", "##ізація"]
apply_wordpiece("токенх", vocab)
# ["[UNK]"]
```

Якщо решта не знаходиться, **return [unk_token]** відкидає й уже знайдений початок. Для порожнього слова цикл не запускається, результат - []. Це greedy longest-match: до коротшого вже вибраного префікса я не повертаюся.

При застосуванні BPE потрібна впорядкована історія злиттів. WordPiece користується фінальним словником і щоразу шукає найдовший фрагмент. Це друга різниця між методами, окрім частоти проти score при навчанні.

7. FLORES, однаковий обсяг і fertility

Я беру **997 паралельних речень × 3 мови = 2991 рядок**. FLORES дає відповідні переклади: один індекс має однаковий зміст. Навчальний список об’єднує всі три мови.

Однакова кількість речень зменшує перевагу мови через більший обсяг прикладів. Паралельні переклади також контролюють зміст. Але це не вирівнює кількість слів, символів, словоформ або частку кожної мови у спільному словнику.

```
def fertility(tokenize, texts):
    texts = list(texts)
    words = sum(len(text.split()) for text in texts)
    if not words:
        raise ValueError("Корпус без слів має давати ValueError")
    return sum(len(tokenize(text)) for text in texts) / words
```

Fertility = загальна кількість токенів / загальна кількість слів. 250 токенів на 100 слів дають 2.5. У знаменнику використовую split(): «кіт,» - одна частина між пробілами, хоча кома може стати окремим токеном.

Я ділю суми всього корпусу. Середнє fertility окремих речень дало б однакову вагу короткому й довгому реченню. Наприклад: 1 слово → 4 токени та 9 слів → 9 токенів. Корпусна fertility = 13/10 = 1.3, а середнє часток = (4 + 1)/2 = 2.5.

Що дає коротший вхід

За доброго покриття нижча fertility дозволяє вмістити більше слів у контекстне вікно. Матриця повної self-attention має розмір T × T, де T - число токенів, тому довжина впливає на витрати. Час усієї моделі не визначається лише цією матрицею.

Один [UNK] теж займає одну позицію. Отже, нижча fertility може бути наслідком втрати інформації. Я перевіряю також UNK, відновлення тексту й результат на нових даних. Для порожнього корпусу функція видає ValueError, щоб не ділити на нуль.

8. Результати: що показують числа

Нижче - **попередній повний запуск** FLORES dev із цієї роботи, збережений у Git (574e69c). У поточному notebook клітинки великого навчання ще без результатів; тут вони повторно не запускалися. Ці числа не подано як новий запуск поточної компактної редакції.

Токенізатор	Англійська	Українська	Грузинська
BPE-500	3.387	4.323	4.928
BPE-1000	2.886	3.668	4.208
BPE-2000	2.469	3.185	3.565
WordPiece-500	4.827	5.945	7.307
WordPiece-1000	4.784	5.810	7.169
WordPiece-2000	4.774	5.412	7.160
GPT-2	1.228	8.024	21.545
GPT-3.5 / GPT-4	1.231	4.202	15.345
GPT-5	1.219	2.442	2.780

Чому BPE зменшує fertility сильніше

BPE зливає найчастіші пари й прибирає багато повторних позицій. Мій WordPiece за нормованим score може вибирати рідкісні фрагменти, які мало впливають на середню довжину. Тому при збільшенні словника BPE дає сильніше скорочення. Це пояснення конкретного досліду, а не універсальна властивість WordPiece.

Для української 500 → 2000: BPE 4.323 → 3.185 (приблизно -26%), WordPiece 5.945 → 5.412 (приблизно -9%). Для грузинської WordPiece 7.307 → 7.160 змінюється зовсім мало.

У попередньому запуску частка вхідних одиниць word_tokenize, що стали [UNK], дорівнювала 13.22% / 18.24% / 19.01% для трьох мов і не змінювалася з розміром словника. Навчання брало тільки слова, а оцінка - також числа й пунктуацію. Нові злиття літер ці одиниці не додають.

Свої моделі оцінено на тому самому dev, де їх навчали: це **train fertility**. GPT має інші навчальні дані, більші словники та зберігає регістр. Порівняння не ізолює лише вплив алгоритму.

9. Українська, англійська та кодування GPT

Чому GPT-2 дає більше токенів українською

Причин кілька: словник GPT-2 краще покриває англійські фрагменти; українська має багатшу словозміну; кирилиця займає кілька байтів UTF-8, і без відповідних злиттів її запис дробиться сильніше. Одними відмінками всю різницю пояснити не можна.

Морфологія: студент, студента, студентом, студентами - різні форми. Відмінок, число, рід і узгодження збільшують різноманіття форм у корпусі. Англійська теж має число, час та інші зміни, проте її система відмінкових форм значно менша. Тому неправильно казати, що в англійській зовсім немає граматичних форм.

Письмо й байти: ASCII-літера зазвичай має 1 байт, українська - 2, грузинська - 3. Якщо в словнику бракує корисних злиттів, потрібні дрібніші фрагменти. Два байти не дорівнюють автоматично двом токенам: злиття може охопити і кілька літер, і частину символу.

Назва у лабораторній	Encoding	Що означає збіг
GPT-2	gpt2	Окреме старіше кодування
GPT-3.5 / GPT-4	cl100k_base	Однаковий encoding дає однакові ID для того самого тексту
GPT-5	o200k_base	У цьому досліді коротші послідовності для всіх трьох мов

Відповідності перевірені за поточним tiktoken та кодом лабораторної. Це конкретні імена моделей у викликах encoding_for_model, а не твердження про всі варіанти сімейств GPT. [2]

Що можна сказати про результати

GPT-2: 1.228 токена на слово англійською, але 8.024 українською й 21.545 грузинською. У цьому наборі GPT-5 має 1.219 / 2.442 / 2.780. Це показує коротше представлення цих текстів у відповідних кодуваннях.

Я не роблю висновку, що модель краще відповідає чи знає мову лише через меншу fertility. Тут немає тесту генерації: tiktoken локально повертає ID, API-запити до моделей не виконуються.

10. Короткі відповіді та порівняння

Метод	Ключове правило	Обмеження
Слова	Цілий токен → ID	Нове слово → <unk>; багато словоформ
N-грами	Вікно n символів із перекриттям	Коротке слово може дати []; порядок втрачається при set
BPE	Найчастіша пара; потім порядок злиттів	Мій символьний варіант обмежений алфавітом
WordPiece	Score при навчанні; greedy при застосуванні	Невдала частина → [UNK] цілого слова
GPT / tiktoken	Готове байтове кодування зі злиттями	Довжина залежить від мови та словника

Що таке fertility?

Середня кількість токенів на слово: ділю загальну кількість токенів на загальну кількість слів. Менше - коротший вхід, але треба перевіряти, чи це не через [UNK].

BPE і WordPiece - у чому дві різниці?

При навчанні: частота проти нормованого score. При застосуванні: історія злиттів проти найдовших фрагментів із готового словника.

Чому більше слів у словнику не завжди сильно допомагає?

Важливо, які саме фрагменти додано. Рідкісні частини мало змінюють середнє; нові злиття відомих літер не усувають усі невідомі одиниці.

«Кіт» і шестиграми?

З маркерами має 5 символів, вікно 6 не вміщується: []. Я доповнив би рядок одним спеціальним символом до довжини 6. Таке саме правило потрібне при навчанні й кодуванні.

Навіщо по 997 речень?

Щоб мова не домінувала лише через число прикладів; паралельні переклади вирівнюють зміст. Слова, символи й частки словника від цього не стають однаковими.

Джерела й прив’язка до роботи

Код: локальний «лаб 2.ipynb» станом на 23.10.2026. Таблиця: попередній запуск у коміті 574e69c. Поточний notebook і його результати не змінювалися.

[1] Hugging Face: WordPiece · BPE
[2] OpenAI tiktoken: відповідності моделей і encoding
[3] FLORES: корпус для експерименту

11. Padding: точне правило та крайові випадки

Поточна char_ngrams повертає [] для надто короткого слова. Нижче - окремий приклад того, як можна додати padding. Це пояснення можливого виправлення; код лабораторної не змінено.

```
def padded_ngrams(word, n=3):
    if not isinstance(n, int) or isinstance(n, bool) or n <= 0:
        raise ValueError("n має бути додатним цілим")
    if not word:
        return []
    bounded = "^" + word + "$"
    bounded = bounded.ljust(n, "~")
    return [bounded[i:i+n]
            for i in range(len(bounded) - n + 1)]
```

Слово та n	Після меж і padding	Результат
кіт, n=3	^кіт$	^кі, кіт, іт$
кіт, n=6	^кіт$~	Одна шестиграма
кіт, n=7	^кіт$~~	Одна семиграма
кіт, n=8	^кіт$~~~	Одна восьмиграма

Кількість доданих елементів: **p = max(0, n - (L + 2))**, де L - довжина непорожнього слова. Після мінімального padding маю хоча б одне положення вікна. Для довгого слова нічого не додаю.

Що важливо уточнити

Один символ, а не напис. Рядок <pad> має п’ять символів Python. Якщо просто дописати його, символьне вікно почне різати й ці літери. У прикладі ~ є одним елементом і має бути зарезервованим: якщо він дозволений у вхідному тексті, потрібна інша домовленість або екранування.

Дві різні задачі padding. Тут я подовжую слово для n-грам. Padding батчу доповнює вже готові послідовності ID до спільної довжини; там зазвичай потрібна маска, щоб модель ігнорувала доповнення. Це не одна операція.

Однакове правило. Якщо навчання не бачило padded-фрагментів, додавання padding лише при застосуванні не гарантує відсутності OOV. Також треба явно визначити порожній рядок: у прикладі він лишається [], щоб не вигадувати слово з одних маркерів.

Це мінімальне праве доповнення до n. Додавання n-1 маркерів з обох боків - інша схема: вона створює більше прикордонних вікон. Не слід перемикатися між схемами без перенавчання словника.

12. BPE: скільки разів можна зливати пари

При навчанні я задаю бюджет num_merges. Алгоритм виконує не більше стількох кроків і може завершитися раніше, якщо сусідніх пар уже немає. Готову модель можна застосовувати до будь-якої кількості текстів: це не витрачає злиття й не перенавчає словник.

Один крок - не одне входження

Один крок вибирає **один тип пари**, а merge_pair зливає всі його неперекривні входження в кожному слові. Якщо пара є тисячу разів, це все ще одне правило у merges. В одному слові кілька входжень можуть скоротитися за один крок.

```
("м", "а", "м", "а", "</w>")
# після одного правила ("м", "а"):
("ма", "ма", "</w>")
```

До злиття - 5 частин, після - 3. Наступні кроки можуть злити «ма» + «ма», а потім «мама» + </w>. Для цього прикладу слово стає одним токеном за 3 правила, хоча спочатку мало 5 частин.

Де природна межа

Для одного слова з K початкових частин потрібно не більше K-1 успішних послідовних скорочень, щоб отримати одну частину. Загальний бюджет для корпусу залежить від усіх різних слів і спільності пар. Якщо S0 - сума довжин початкових послідовностей різних слів, U - їх кількість, то S0-U є верхньою межею кількості успішних кроків у цій схемі: кожен крок скорочує хоча б одну позицію. Зазвичай кроків менше, бо правило спрацьовує багато разів.

Бюджет словника

У лабораторній початковий розмір V0 - число різних символів плюс </w>. Для цілі V я задаю **M = V - V0**. Наприклад, якщо V0 = 100, для V = 500 беру 400 злиттів. Це умовний приклад, а не виміряний алфавіт цієї роботи.

«Одне злиття додає один токен» - корисне спрощення. Якщо результат уже є в наборі або пари закінчилися, кількість унікальних токенів може не зрости до цілі. Тому фактичний словник варто перевіряти через set. Історія правил, поточне розбиття навчальних слів і повний словник - різні об’єкти.

13. BPE: розбір навчального циклу

```
def train_bpe(texts, num_merges):
    if not isinstance(num_merges, int) or num_merges < 0:
        raise ValueError("num_merges має бути невід'ємним цілим")
    vocabulary = {tuple(word) + ("</w>",): count
                  for word, count in word_frequencies(texts).items()}
    merges = []
    for _ in range(num_merges):
        pair = most_frequent_pair(vocabulary)
        if pair is None:
            break
        updated = Counter()
        for symbols, count in vocabulary.items():
            updated[merge_pair(symbols, pair)] += count
        vocabulary = updated
        merges.append(pair)
    return merges
```

word_frequencies прибирає числа й пунктуацію з навчання та рахує повтори слів. Ключ vocabulary - tuple частин, значення - частота. Кожен обхід зовнішнього циклу вибирає пару, а внутрішній перебудовує всі слова.

Counter додає частоти до оновленого ключа; **pair is None** завершує цикл без пар. **merges.append(pair)** зберігає саме правило й порядок, а не фінальні токени кожного слова. Це наївна реалізація з повторним повним підрахунком.

Час і повторне використання

Нехай S - початкова кількість частин у різних словах, M - число правил. Груба межа навчання - O(M×S) операцій над частинами, без урахування довжини рядків і службових витрат Python. Повтори однакових слів враховано вагами, а не копіями. Для нового слова довжини L apply_bpe робить M проходів: грубо O(M×(L+1)).

Тому великий корпус і багато злиттів можуть довго працювати й на CPU. GPU для цієї Python-реалізації сам по собі не допоможе: код не переносить підрахунки на GPU. Кеш розбиття повторних слів прискорив би застосування, не змінюючи правил.

14. WordPiece: чому інтуїція може підвести

Більший словник і greedy

Більший словник дає більше доступних частин, але greedy не перебирає всі розбиття. Доданий довгий початок може змінити шлях і привести до невідомого продовження. Тому для кожного окремого нового слова не можна обіцяти монотонне покращення.

```
old = {"[UNK]", "a", "##bc", "##d"}
apply_wordpiece("abcd", old)
# ["a", "##bc", "##d"]

new = old | {"ab"}
apply_wordpiece("abcd", new)
# ["[UNK]"]
```

Новий словник змушує алгоритм почати з «ab». Потрібної частини ##c або ##cd немає, тому повертається [UNK]. Старий шлях a + ##bc + ##d досі існує, але повернення до нього в коді немає. Це приклад поведінки саме такого greedy-пошуку.

Три речі, які легко переплутати

не означає окремий символ тексту. Це службова позначка позиції частини. «a» на початку й «##a» всередині - різні записи словника. При злитті зайвий внутрішній ## прибирається, а позиція отриманого фрагмента зберігається.

Score не дорівнює частоті. Його знаменник змінює пріоритет. Це пояснює, чому багато нових записів можуть охоплювати небагато повторень у корпусі. Формулу не слід називати прямою ймовірністю або точним навчанням усіх промислових WordPiece.

Один [UNK] може приховувати довге слово. Тому падіння fertility без оцінки покриття не доводить покращення. У лабі цифри й пунктуація беруть участь в оцінці, але не в навчанні власних підсловних словників. Збільшення числа злиттів не робить ці умови однаковими.

Які перевірки я б показав

Відоме слово; нове слово з відомих символів; невідомий символ; порожній рядок; слово, для якого можливі кілька розбиттів. Для WordPiece я перевіряю результат цілого слова, а не лише те, що вдалося знайти на початку.

15. Як коректно пояснювати експеримент

1. Навчання й застосування - окремі етапи

Навчання визначає словник або правила злиття. Застосування перетворює текст за готовими правилами. Мої train_bpe і train_wordpiece навчають токенізатори; вони не навчають embedding чи трансформер. tiktoken узагалі бере готове кодування.

2. Коротші рядки не завжди кращі

Я перевіряю fertility разом із покриттям, збереженням тексту, часом і пам’яттю. Мій lower() втрачає регістр, а word_tokenize не повертає пробіли. Навіть якщо всі слова розбиті без UNK, це ще не гарантує точного відновлення початкового тексту.

3. Однакова кількість речень - контроль, а не повна рівність

Паралельні речення контролюють зміст і число прикладів, але не довжину перекладу. Мова з більшою кількістю символів або різних форм усе одно може сильніше впливати на підрахунки. Не можна казати, що кожна мова точно отримала третину словника.

4. Новий корпус і перевірка узагальнення

Train fertility оцінює представлення знайомого корпусу. На окремих текстах можуть змінитися теми, імена, письмо та частоти фрагментів. Коректний наступний крок - зафіксувати словник на навчальних даних і порівняти його на іншому наборі без перенавчання.

5. Нюанс про «символ»

Python len() для рядка рахує Unicode-кодові точки, а не байти й не завжди видимі символи. Літера з комбінованим наголосом або складний emoji можуть містити кілька таких елементів. Мої n-грами працюють саме зі зрізами Python-рядка; байтовий GPT-токенізатор працює на іншому рівні.

6. Пам’ятка відповіді на уточнення

Що змінюю? Правило розбиття, словник чи лише спосіб оцінки? На яких даних навчаю й на яких перевіряю? Чи збережений текст? Чи не знизилося число токенів лише через [UNK]? Ці питання допомагають пояснювати результат, а не просто називати менше число кращим.

Усі додаткові приклади - навчальні пояснення. Вони не були вставлені в notebook і не змінюють наведені результати попереднього запуску. Джерела та прив’язка таблиці до Git наведені на сторінці 10.